

Guia de Defensa Tecnica - EcoTrip 2026

1. Resumen rapido para explicar al profesor

EcoTrip 2026 es una aplicacion Android nativa hecha con Kotlin y Jetpack Compose. Tiene dos pantallas: una pantalla de configuracion del viaje y una pantalla de resumen tipado. La arquitectura separa el modelo de datos, la persistencia local, el ViewModel y la interfaz declarativa.

La idea central es evitar perdida de datos ante recomposicion, rotacion de pantalla, muerte del proceso y cierre completo de la app. Para eso se usan tres niveles de resiliencia:

- ViewModel: mantiene estado mientras el proceso sigue vivo.
- SavedStateHandle: restaura campos si Android destruye y recrea la actividad.
- DataStore: guarda preferencias en disco para recuperarlas despues de cerrar totalmente la app.

2. Archivos principales y que hace cada uno

MainActivity.kt

Ubicacion: `app/src/main/java/com/example/programacionmovil/MainActivity.kt`

Es el punto de entrada de la app. Activa `enableEdgeToEdge()`, crea el `EcoTripViewModel` usando su `Factory` y carga la interfaz con `setContent`. Dentro de `setContent` se aplica `ProgramacionMovilTheme` para que toda la app use Material 3 y color dinamico.

Si el profesor pregunta por que no hay XML: porque toda la UI se monta desde Compose con `setContent`, sin `findViewById` ni layouts tradicionales.

PreferenciaViaje.kt

Ubicacion: `app/src/main/java/com/example/programacionmovil/data/PreferenciaViaje.kt`

Contiene el modelo principal pedido por el PDF:

- `nombreViajero`
- `destino`
- `diasDuracion`
- `tipoTransporte`
- `soloRutasBajaHuella`

Tambien define `TipoTransporteEcologico`, un `enum class` con Tren, Bicicleta y Vehiculo electrico. La funcion `generarRecomendacionRuta()` crea un resumen calculado para la pantalla 2: distancia estimada, emisiones, ahorro de CO2 y tramos sugeridos.

Si el profesor pide agregar otro transporte, se cambia aqui: se agrega una entrada nueva al enum y un caso en el `when` de `generarRecomendacionRuta()`.

PreferenciaViajeRepository.kt

Ubicacion: `app/src/main/java/com/example/programacionmovil/data/PreferenciaViajeRepository.kt`

Es la capa de persistencia en disco. Usa Preferences DataStore con el archivo `ecotrip_preferencias`. Expone un `Flow<PreferenciaViaje>` para leer preferencias de forma reactiva y una funcion `guardar()` para escribir valores.

DataStore se usa para preferencias globales permanentes. Si el usuario cierra la app por completo, los datos validos guardados pueden recuperarse en el siguiente inicio.

EcoTripUiState.kt

Ubicacion: `app/src/main/java/com/example/programacionmovil/ui/EcoTripUiState.kt`

Representa el estado que consume Compose. Guarda los textos tal como se ven en pantalla y calcula validaciones:

- `nombreValido`
- `destinoValido`
- `diasValidos`
- `esValido`
- `preferenciaValida`

El punto clave es `diasDuracionTexto.toIntOrNull()`: evita crashes si el campo esta vacio o tiene un valor no numerico.

EcoTripViewModel.kt

Ubicacion: `app/src/main/java/com/example/programacionmovil/ui/EcoTripViewModel.kt`

Es la unica fuente de verdad de la UI. Recibe eventos desde las pantallas y actualiza el `SavedStateHandle`. Expone `uiState` como `StateFlow`.

Responsabilidades:

- Guardar campos temporales en `SavedStateHandle`.
- Filtrar entradas numericas con `value.filter(Char::isDigit)`.
- Restaurar desde DataStore si la app abre sin estado previo.
- Guardar preferencias validas en disco.
- Emitir mensajes de error o exito para el Snackbar.

Si el profesor pregunta por UDF: los eventos suben desde la UI al ViewModel, y el estado baja desde `uiState` hacia Compose.

EcoTripRoutes.kt

Ubicacion: `app/src/main/java/com/example/programacionmovil/navigation/EcoTripRoutes.kt`

Define rutas fuertemente tipadas con `@Serializable`:

- `FormularioViajeRoute`
- `ResumenRutaRoute`

`ResumenRutaRoute` exige parametros estrictos: nombre, destino, dias, transporte y bandera de baja huella. Esto reduce errores porque el compilador verifica el contrato de navegacion.

EcoTripApp.kt

Ubicacion: `app/src/main/java/com/example/programacionmovil/ui/EcoTripApp.kt`

Contiene el `NavHost` . La pantalla inicial es `FormularioViajeRoute` . Al planificar, navega a `ResumenRutaRoute` pasando argumentos tipados.

Tambien aplica higiene del Back Stack con:

- `launchSingleTop = true`
- `popUpTo<FormularioViajeRoute> { ... }`

Esto evita acumulacion de pantallas duplicadas.

EcoTripScreens.kt

Ubicacion: `app/src/main/java/com/example/programacionmovil/ui/EcoTripScreens.kt`

Contiene las dos pantallas Compose.

`FormularioViajeScreen` usa:

- `Scaffold`
- `CenterAlignedTopAppBar`
- `FloatingActionButton`
- `OutlinedTextField`
- `FilterChip`
- `Switch`
- `SnackbarHost`

`ResumenRutaScreen` recibe la ruta tipada, reconstruye una `PreferenciaViaje` y muestra la configuracion formateada. Usa colores semanticos de `MaterialTheme.colorScheme` , por eso responde al color dinamico de Android.

Theme.kt y Color.kt

Ubicacion: `app/src/main/java/com/example/programacionmovil/ui/theme/`

`Theme.kt` activa Material You con:

- `dynamicLightColorScheme(context)`
- `dynamicDarkColorScheme(context)`

Los colores de `Color.kt` solo son fallback para dispositivos sin color dinamico. La UI no depende de hexadecimales directos, sino de roles semanticos como `primaryContainer` , `secondaryContainer` y `tertiaryContainer` .

3. Checklist de cumplimiento contra el PDF

- compileSdk 36 y targetSdk 36: cumplido en `app/build.gradle.kts` .
- Java/JBR 21: cumplido con `sourceCompatibility` , `targetCompatibility` , `jvmTarget` y `org.gradle.java.home` .
- Catalogo de versiones: cumplido en `gradle/libs.versions.toml` .
- Jetpack Compose sin XML de vistas: cumplido.
- Dos pantallas principales: cumplido.
- Scaffold, TopAppBar y FAB: cumplido.
- Modelo `PreferenciaViaje` inmutable con `val` : cumplido.

- Tipo de transporte ecologico: cumplido con `TipoTransporteEcologico`.
- Switch de baja huella: cumplido con `soloRutasBajaHuella`.
- ViewModel como fuente unica de verdad: cumplido.
- SavedStateHandle para estado temporal: cumplido.
- DataStore para preferencias permanentes: cumplido.
- Navegacion type-safe con `@Serializable`: cumplido.
- `popUpTo` y `launchSingleTop`: cumplido.
- Material 3 y color dinamico: cumplido.
- Validacion segura con `toIntOrNull()`: cumplido.
- Pruebas unitarias basicas: cumplido con `EcoTripStateTest`.

Pendiente operativo fuera del codigo:

- Ejecutar en emulador API 36.
- Tomar capturas para el informe.
- Cambiar wallpaper y capturar Material You.
- Subir a GitHub con minimo 4 commits reales de los integrantes.
- Exportar el informe final a PDF con portada, capturas, bibliografia APA y anexo de codigo.

4. Donde podria pedir cambios el profesor

Cambio 1: Agregar un nuevo campo al formulario

Ejemplo: agregar `origen` o `presupuesto`.

Archivos a tocar:

- `PreferenciaViaje.kt`: agregar propiedad con `val`.
- `EcoTripUiState.kt`: agregar estado, validacion y construccion de `PreferenciaViaje`.
- `EcoTripViewModel.kt`: agregar clave en `SavedStateHandle` y funcion `actualizar...`.
- `PreferenciaViajeRepository.kt`: agregar key de DataStore.
- `EcoTripScreens.kt`: agregar `OutlinedTextField`.
- `EcoTripRoutes.kt` y `EcoTripApp.kt`: agregar parametro si debe viajar al resumen.

Cambio 2: Cambiar rango de dias validos

Ejemplo: de 1..30 a 1..15.

Archivo principal:

- `EcoTripUiState.kt`

Cambiar:

```
```kotlin
```

```
diasDuracion?.let { it in 1..30 } == true
```

```
```
```

por:

```
```kotlin  

diasDuracion?.let { it in 1..15 } == true

```
```

Tambien conviene actualizar el texto de error en `EcoTripScreens.kt`.

Cambio 3: Agregar otro transporte ecologico

Ejemplo: `BusElectrico`.

Archivos:

- `PreferenciaViaje.kt` : agregar entrada al enum.
- `generarRecomendacionRuta()` : agregar caso al `when`.

La UI se actualiza automaticamente porque recorre `TipoTransporteEcologico.entries`.

Cambio 4: Cambiar la navegacion para pasar un objeto completo

Actualmente se pasan parametros estrictos en `ResumenRutaRoute`. Si pide pasar objeto completo, la ruta podria recibir un JSON serializado o una clase serializable con todos los campos. La explicacion tecnica seria que ambas formas son type-safe si el contrato esta definido con clases `@Serializable`.

Archivos:

- `EcoTripRoutes.kt`
- `EcoTripApp.kt`
- `EcoTripScreens.kt`

Cambio 5: Agregar boton para limpiar formulario

Archivos:

- `EcoTripViewModel.kt` : crear `limpiarFormulario()`.
- `EcoTripScreens.kt` : agregar boton en `TopAppBar` o `FAB` secundario.

La funcion debe poner claves del `SavedStateHandle` en valores vacios y restaurar defaults.

Cambio 6: Modificar persistencia para guardar solo preferencias globales

Si el profesor es estricto con "DataStore solo preferencias globales", se puede dejar en `DataStore` solamente `tipoTransporte` y `soloRutasBajaHuella`, mientras `SavedStateHandle` conserva destino y duracion.

Archivo:

- `PreferenciaViajeRepository.kt`

Eso no rompe la arquitectura: `DataStore` queda para disco permanente y `SavedStateHandle` para estado temporal de formulario.

Cambio 7: Demostrar Material You

Si pide cambiar colores hardcoded, responder que la app usa `MaterialTheme.colorScheme`. Los lugares principales estan en:

- `Theme.kt`
- `EcoTripScreens.kt`

No se deben poner colores hexadecimales directamente en las pantallas.

5. Respuestas cortas para defensa oral

****Que diferencia hay entre ViewModel, SavedStateHandle y DataStore?****

ViewModel mantiene estado en memoria mientras el proceso vive. SavedStateHandle sobrevive recreaciones y muerte de proceso porque guarda un bundle de estado. DataStore persiste en disco y mantiene datos aun si la app se cierra por completo.

****Que es UDF?****

Flujo Unidireccional de Datos: la UI envia eventos al ViewModel y el ViewModel emite estado hacia la UI. La pantalla no decide la logica de negocio.

****Por que Navigation type-safe reduce crashes?****

Porque los destinos se declaran como clases serializables con parametros obligatorios. Si falta un parametro o tiene tipo incorrecto, el error aparece antes o queda localizado en compilacion/desarrollo, no como una clave String mal escrita en runtime.

****Por que se usa toIntOrNull()?****

Porque un campo numerico puede estar vacio o contener datos invalidos. `toIntOrNull()` devuelve null en vez de lanzar excepcion, evitando cierres forzados.

****Por que usar MaterialTheme.colorScheme?****

Porque conecta la interfaz con los roles semanticos de Material 3 y permite que Android aplique color dinamico segun el wallpaper.

6. Comandos de verificacion

```
```powershell
```

```
.\gradlew.bat testDebugUnitTest
```

```
.\gradlew.bat assembleDebug
```

```
```
```

La compilacion debe terminar con `BUILD SUCCESSFUL`.